

Sparse Coding and Autoencoders: Transformative Approaches in Digital Image Processing with Applications in Multimedia, Remote Sensing, and Computer Vision, Including an In-Depth Exploration of their Role in Handwritten Digit Recognition.

Million Zere Solomon

Department of Electrical and Computer Engineering
Toronto Metropolitan University, Toronto, ON, CA
million.solomon@torontomu.ca

Abstract—In This paper I explored the transformative potential of sparse coding and autoencoders in the realm of digital image processing. These techniques have demonstrated remarkable efficacy in various applications, including multimedia, remote sensing, and computer vision. The paper delves into the underlying principles of sparse coding and autoencoders, explaining their role in extracting meaningful representations from complex image data. Furthermore, it investigates their specific applications, showcasing how these techniques enhance image processing tasks in multimedia environments, contribute to precise analysis in remote sensing applications, and bolster the capabilities of computer vision systems. The paper also includes a detailed discussion on the application of sparse coding and autoencoders in handwritten digit recognition by comparing and evaluating three methods proposed by different authors for handwritten digit recognition. In the first method the authors propose a KNN-based architecture using supervised machine learning, while the second method proposed by different writers uses sparse coding and a local maximum operation for handwritten digit recognition, A third method explores the use of three popular Autoencoder variants: sparse AEs (SAEs), denoising AEs (DAEs), and contractive AEs (CAEs) in solving the problem of handwritten digits classification. Through a comprehensive exploration of these methodologies, this paper illuminates the transformative impact of sparse coding and autoencoders on advancing digital image processing techniques and their wide-ranging applications.

Index Terms— Classification, Contractive auto-encoder (CAE), Denoising auto-encoder (DAE), Feature Extraction, Handwritten digit recognition, KNN, Neural Networks, Sparse Coding, Support Vector machine (SVM), Sparse Auto-encoder (SAE).

I. INTRODUCTION

T

HE techniques of autoencoders and Sparse coding are important in working with images, helping a lot in computer

vision, medical imaging, and signal processing [1]. Sparse coding, emphasizing the representation of data with a minimal number of non-zero coefficients, proves especially valuable in handling high-dimensional data, effectively reducing redundancy and enhancing processing efficiency. Autoencoders, neural network architectures designed for unsupervised learning, encode input data into a compact representation and then reconstruct the original input. Successful in tasks like image denoising, compression, and reconstruction, the combination of sparse coding and autoencoders is synergistic, facilitating more effective representation learning by incorporating sparsity constraints into the encoding process. This integration enhances feature interpretability and contributes to improved performance in various image-related applications. In medical imaging, these techniques assist in tasks like anomaly detection and image segmentation. In computer vision applications, including object recognition and image classification, the significance of sparse coding and autoencoders remains pivotal. They push the boundaries of what is achievable in extracting meaningful information from complex visual data.

II. BACKGROUND

Digital image processing is the use of a digital computer to process digital images [2]. Digital images are composed of pixels, which are small dots of color or intensity. Each pixel has a value that represents its color or intensity. For example, a binary image has only two values: 0 for black and 1 for white. A grayscale image has values from 0 to 255, where 0 is black, 255 is white, and the values in between are shades of gray. A color image has three values for each pixel: one for red, one for green, and one for blue.

The basic steps involved in digital image processing are:

- Image acquisition: This involves capturing an image using a digital camera or scanner, or importing an existing image into a computer.
- Image enhancement: This includes improving the

visual quality of an image, such as increasing contrast, reducing noise, and removing artifacts.

- Image restoration: It encompasses removing degradation from an image, such as blurring, noise, and distortion.
- Image segmentation: It involves dividing an image into regions or segments, each of which corresponds to a specific object or feature in the image.
- Image representation and description: This comprises representing an image in a way that can be analyzed and manipulated by a computer, and describing the features of an image in a compact and meaningful way.
- Image analysis: It consists of using algorithms and mathematical models to extract information from an image, such as recognizing objects, detecting patterns, and quantifying features.
- Image synthesis and compression: This involves generating new images or compressing existing images to reduce storage and transmission requirements.

Sparse coding and autoencoders are two techniques that can be used for image representation and compression. Sparse coding is a method of finding a set of basis vectors that can represent an image using only a few nonzero coefficients. This reduces the dimensionality and redundancy of the image data. Autoencoders are neural networks that learn to encode an input image into a lower-dimensional representation and then decode it back to the original image. The goal is to minimize the reconstruction error between the input and output images. Both sparse coding and autoencoders can capture the essential features of an image and reduce its size.

Sparse coding and autoencoders can also be used for image analysis and synthesis. For example, sparse coding can be used to detect edges, corners, and other salient features in an image. Autoencoders can be used to generate new images that are similar to the input images, such as faces or handwritten digits. They can also be used to modify existing images, such as adding color or changing style.

III. SPARSE CODING FOR IMAGE PROCESSING

Sparse coding is a way of making images smaller and simpler by using fewer pieces of information. These pieces are called atoms and they are like building blocks that can make different shapes when you put them together. The atoms are stored in a dictionary, which is like a book that has all the possible atoms you can use. Sparse coding tries to find the best atoms for each image and the best way to combine them.

Sparse coding is useful for image processing because it can help with tasks like removing noise, finding features, and compressing data. For example, if an image has some unwanted dots or scratches, sparse coding can find a cleaner version of the image by using only the atoms that match the important parts of the image. Sparse coding can also help with

finding features, which are parts of the image that have some meaning or pattern. For example, sparse coding can find edges, corners, or shapes in an image by using atoms that look like them. Sparse coding can also help with compressing data, which means making the image smaller without losing too much quality. For example, sparse coding can reduce the amount of information needed to store or send an image by using only a few atoms and their combinations instead of all the pixels.

Sparse coding is related to other methods which try to find simple and meaningful representations of data. Sparse coding is different because it focuses on finding representations that are sparse, which means that only a few atoms are used for each image and most of them are zero or very small. This makes the representation more efficient and easier to interpret.

Some examples of applications that use sparse coding are face recognition, object recognition, image restoration, and super-resolution. Face recognition is a task where you try to identify a person from their face image. Sparse coding can help with this by finding atoms that represent different facial features and expressions. Object recognition is a task where you try to identify what kind of object is in an image. Sparse coding can help with this by finding atoms that represent different parts of objects and their shapes. Image restoration is a task where you try to improve the quality of an image that has some damage or distortion. Sparse coding can help with this by finding atoms that match the original image and removing the ones that cause noise or blur. Super-resolution is a task where you try to increase the resolution of an image, which means making it bigger and clearer. Sparse coding can help with this by finding atoms that add more details and sharpness to the image.

IV. AUTOENCODERS FOR IMAGE PROCESSING

Autoencoders are a type of artificial neural network that can learn to compress and reconstruct data [3]. They have two parts: an encoder and a decoder. The encoder takes an input image and transforms it into a smaller code representation. The decoder takes the code and tries to recreate the original image as closely as possible. This way, autoencoders can reduce the dimensionality of the data and also learn useful features from it.

Autoencoders have many applications in image processing, such as:

- Image compression: Autoencoders can reduce the size of an image by encoding it into a smaller code and then decoding it back to an image. This can save storage space and bandwidth for transmitting images.
- Image denoising: Autoencoders can remove noise from an image by learning to reconstruct a clean image from a noisy one. This can improve the quality and clarity of the image.
- Feature extraction: They can learn to extract important features from an image that can be used for other tasks, such as classification, detection, or

recognition. For example, autoencoders can learn to recognize faces or objects in an image.

- Image generation: Autoencoders can generate new images that are similar to the input data by randomly sampling codes from the latent space and decoding them into images. This can be used for creating realistic or artistic images.
- Sequence to sequence prediction: They can learn to predict the next frame in a video sequence by encoding the current frame and decoding the next one. This can be used for video analysis or synthesis.

V. APPLICATION IN MULTIMEDIA PROCESSING

One of the applications of sparse coding and autoencoders is multimedia image processing, where the goal is to compress images efficiently and preserve their quality [4]. Both methods aim to reduce the redundancy and noise in the image data, but they have different advantages and disadvantages.

Sparse coding can produce visually superior reconstructed images, as it preserves more content-relevant information and higher frequencies. However, sparse coding is computationally expensive and requires solving an optimization problem for each image. Autoencoders can achieve faster compression and reconstruction, as they use feed-forward neural networks that can be trained end-to-end. However, autoencoders may lose some details and introduce artifacts in the reconstructed images, especially for small images with high resolution.

VI. APPLICATION IN BIOMEDICAL IMAGING

In biomedical imaging, sparse coding and autoencoders have been widely adopted for various applications due to their ability to efficiently represent and analyze complex medical data. One of the primary applications of sparse coding is in image denoising, where it is used to remove unwanted noise from medical images. This technique has been successfully applied to various medical imaging modalities, including magnetic resonance imaging (MRI), positron emission tomography (PET), and computed tomography (CT) scans.

Another important application of sparse coding is in image segmentation, where it helps to accurately distinguish between different structures within an image. By exploiting the spatial coherence of image patterns, sparse coding can effectively separate signals from different tissues or structures, even when their intensity values are similar. This has proven particularly useful in tumor detection and characterization, where precise segmentation is crucial for diagnosis and treatment planning.

One key advantage of using autoencoders in biomedical imaging is their ability to capture sophisticated patterns and relationships in the data that might be difficult to detect through traditional methods. By learning hierarchical representations of the input data, autoencoders can reveal underlying structures and features that are not immediately apparent from the raw image data. This has led to successful applications in image classification and disease diagnosis.

Furthermore, the encoded representation produced by

autoencoders can serve as a compact descriptor for image retrieval and matching. This has been particularly useful in medical imaging, where the ability to quickly search for and match images with specific characteristics is essential for tasks such as tumor tracking, disease progression modeling, and image-guided therapy.

In summary, sparse coding and autoencoders have revolutionized the field of biomedical imaging by providing efficient and effective ways to represent, analyze, and process complex medical data. From denoising and segmentation to dimensionality reduction and feature extraction, these techniques have enabled researchers and clinicians to gain valuable insights into the structure and function of biological systems, ultimately contributing to better diagnostic and therapeutic strategies.

VII. APPLICATION IN REMOTE SENSING AND COMPUTER VISION

Remote sensing and computer vision are fields that involve the acquisition and analysis of vast amounts of image data [5]. Efficiently processing and extracting valuable information from these data can be challenging due to the large volume and complexity of the information. In remote sensing, sparse coding has been applied to tasks such as land cover classification, crop type mapping, and object detection.

Autoencoders have been successfully applied to various computer vision tasks, such as image denoising, deraining, and super-resolution. In remote sensing, autoencoders have been used for image fusion, where they combine data from multiple sensors or different spectral bands to generate higher-quality images.

The combination of sparse coding and autoencoders has also shown promising results in remote sensing and computer vision. By integrating sparse coding with autoencoders, one can leverage the strengths of both techniques to achieve better performance and efficiency.

VIII. APPLICATION IN HANDWRITTEN DIGIT RECOGNITION LITERATURE SURVEY

Handwritten digit recognition is a fundamental challenge in pattern recognition and machine learning [6]. Sparse coding has found substantial application in improving the representation of handwritten digits for recognition purposes. By emphasizing essential features while discarding redundant information, sparse coding aids in capturing the distinctive characteristics of individual digits. This is particularly beneficial in scenarios where variability in writing styles and noise in the data pose challenges to accurate recognition. Sparse coding has been successfully implemented to enhance feature extraction and improve the robustness of handwritten digit recognition systems. Autoencoders, with their ability to learn compact representations of data, have been extensively utilized in handwritten digit recognition tasks. The encoder-decoder architecture of autoencoders enables the extraction of salient features from the input digits. In the context of handwritten digit recognition, autoencoders contribute to

dimensionality reduction, enabling a more efficient representation of the input data. This aids in mitigating the curse of dimensionality and facilitates more accurate recognition, especially in scenarios where variations in writing styles can be substantial. The integration of sparse coding and autoencoders has shown promise in further enhancing the performance of handwritten digit recognition systems. By combining the sparsity constraints of sparse coding with the feature learning capabilities of autoencoders, the resulting model exhibits improved interpretability and recognition accuracy. This integration enables the system to efficiently handle variations in writing styles, noise, and other challenges commonly encountered in handwritten digit datasets. The application of sparse coding and autoencoders in handwritten digit recognition extends to real-world scenarios. In financial institutions, these techniques are employed for automatic check processing, enabling the accurate extraction and recognition of handwritten digits on checks. Furthermore, postal services leverage these methods for efficient sorting of mail based on handwritten zip codes, demonstrating the practical significance of these techniques in enhancing automation and accuracy in various domains.

While sparse coding and autoencoders offer substantial benefits in handwritten digit recognition, challenges persist. Variability in writing styles, the presence of noise, and the need for scalability in large datasets pose ongoing challenges. Solutions involve refining model architectures, exploring advanced training strategies, and integrating additional contextual information to improve the robustness of recognition systems. Current trends in the application of sparse coding and autoencoders in handwritten digit recognition include the exploration of deep learning architectures and the integration of contextual information from surrounding digits. Future directions involve leveraging generative models to enhance dataset augmentation and the development of hybrid models combining sparse coding, autoencoders, and other advanced techniques for more comprehensive recognition systems.

The application of sparse coding and autoencoders has significantly advanced the field of handwritten digit recognition. Their synergistic integration enhances the interpretability and robustness of recognition systems, making them valuable tools in various real-world applications. As challenges are addressed and technologies evolve, the continued exploration of these techniques promises to further elevate the accuracy and efficiency of handwritten digit recognition systems.

VIII.1. HANDWRITTEN DIGIT RECOGNITION USING MACHINE LEARNING

[7] The study presents a method for recognizing handwritten digits using K-Nearest Neighbors (KNN) Machine Learning algorithm. Machine Learning (ML) is the scientific study of algorithms and statistical models that computer systems use to effectively perform a specific task without using explicit instructions, relying on patterns and

inference instead. K-Nearest Neighbors (KNN) is a simple, non-parametric, and lazy supervised learning algorithm used for classification. It classifies an unknown sample based on the majority class of its k-nearest neighbors in the feature space. The paper uses the MNIST dataset, which contains 60,000 training set and 10,000 testing set of handwritten digit images from 0 to 9.

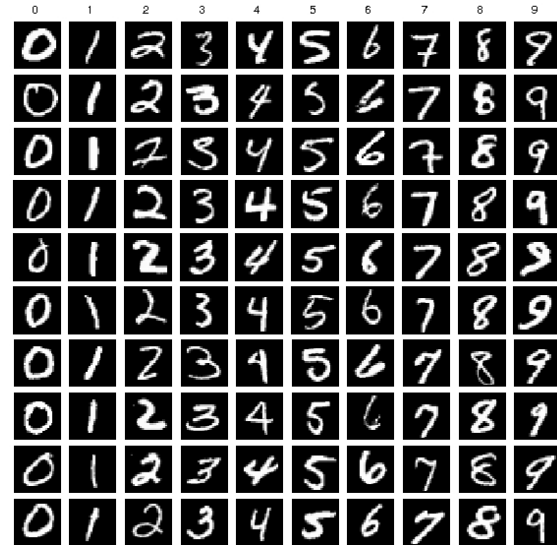


Fig.1 MNIST Dataset (adapted from [7])

The implementation of the KNN algorithm in the system involves several key steps.

- Feature Extraction:
 - Each image is typically represented as a vector of pixel values. For the MNIST dataset, this would be a 28x28 image flattened into a 784-dimensional vector.
- Training:
 - The training phase involves storing the feature vectors and their corresponding class labels in memory.
- Prediction:
 - To classify a new digit, the algorithm calculates the distance between the input feature vector and all the training feature vectors. Common distance metrics used is Euclidean distance.
 - Let x be the input feature vector and X_i be the feature vector of the i -th training example. The distance is calculated as follows:

$$\text{distance}(x, X_i) = \sqrt{\sum_{j=1}^d (x_j - X_{ij})^2}$$

..... (1)

where d is the number of features.

- Once distances are calculated, the algorithm identifies the k training examples with the shortest distances (nearest neighbors).

- The class of the new digit is determined by a majority vote among its k nearest neighbors.

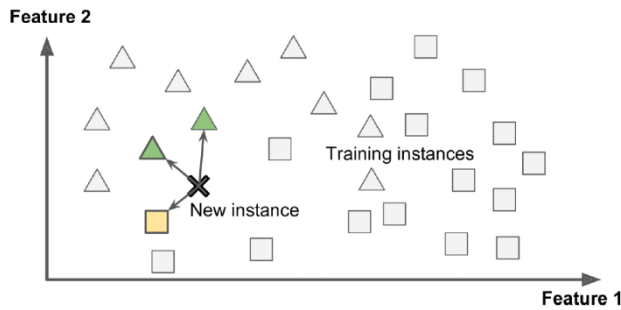


Fig 2 Instance based learning of KNN Algorithm (adapted from “what is instance-based machine learning”, Sandhya Agrawal)

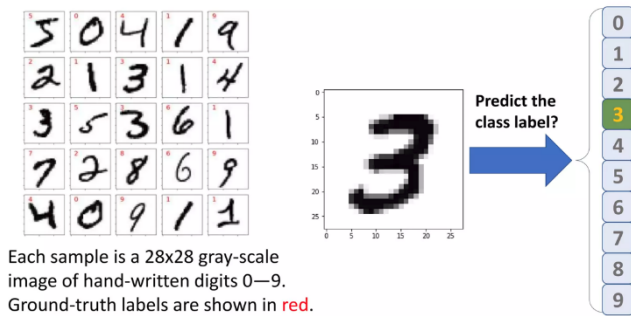


Fig 3. Hand written digit classification

The proposed study found that the algorithm has predicted with accuracy and needed less computation. But with some strengths and weaknesses.

- Strengths:
 - Simple to understand and implement.
 - No training phase; it's a lazy learner.
 - Can handle multiclass classification.
- Weaknesses:
 - Computationally expensive during the prediction phase, especially for large datasets.
 - Sensitive to irrelevant features.
 - The choice of k is critical and might need to be tuned based on the specific dataset.

Their conclusion is that KNN provides a straightforward approach for handwritten digit recognition, especially in scenarios where the dataset is not very large. However, its computational cost can be a limiting factor when dealing with high-dimensional datasets like images. In practice, it may not be the first choice for image-based tasks, where more complex models like Convolutional Neural Networks (CNNs) often outperform KNN.

VIII.2. SIMPLE METHOD FOR HIGH-PERFORMANCE DIGIT RECOGNITION BASED ON SPARSE CODING

[8] The study proposes a method of feature extraction for handwritten digit recognition using sparse-coding and a local maximum operation. The authors employed the unsupervised sparsenet algorithm to learn a basis for representing patches of handwritten digit images. Then they used the basis to extract local coefficients. In their next step, they applied a local maximum operation to implement local shift invariance. As this method gave them a new representation of the digits. They fed this resulting feature vector to a support vector machine (SVM) classifier to obtain classification performance for digit recognition using the Modified National Institute of Standards and Technology (MNIST) dataset.

Their method for feature extraction can be represented with two steps:

1. Compute a number of coefficient images Using learned basis.
2. Apply a local maximum operation.

1. LEARNING THE BASIS FUNCTIONS AND OBTAINING COEFFICIENT IMAGES

First to improve the convergence of the Sparsenet algorithm they applied preprocessing to the image patches.

They obtained the centered patch vector $P(x, y)$ such that

$$P(x, y) = \tilde{P}(x, y) - \bar{P} \quad \dots (2)$$

Where:

- $\bar{P}$ refer to the mean of a large number of such $\tilde{P}(x, y)$ given by $\tilde{P}(x, y) = \hat{P}(x, y) - \bar{\hat{P}}(x, y)$, which are obtained from training images with $\tilde{P}(x, y)$ placed at random positions.
 - o If I denote an entire image. $\hat{P}(x, y)$ denotes a vector containing all N^2 pixel values of an image patch of size $N \times N$ centered at position (x, y)
 - o $\bar{\hat{P}}(x, y)$ denotes the mean value of the pixel entries $\hat{P}(x, y)$

As this Patch vector $P(x, y)$ of a given image I can be obtained from a linear combination of K number of basis functions $\vec{w}_j$ of size N^2 with coefficients $\vec{a}$ such as:

$$P(x, y) = \sum_{j=1}^K \vec{w}_j a_j + \vec{\epsilon} = W\vec{a} + \vec{\epsilon} \quad \dots (3)$$

They introduced $\vec{\epsilon}$ to correspond the assumption of small noise that leads to a certain error.

Then using a Sparsenet algorithm that uses optimization problem:

$$\min_W E \left(\min_{\vec{a}} (\|P(x, y) - W\vec{a}\| + \lambda S(\vec{a})) \right) \quad \dots (4)$$

Where $S(\vec{a})$ is regularization term and λ is the strength of the

penalty.

They were able to obtain sparse coefficients $\vec{a}$ and basis functions $\vec{w}_j$ using training data.

On their next step they obtained the coefficient images F_j which contains the coefficients a_j of the basis functions $\vec{w}_j$ for all patches of the input image. The sparsenet coefficients were obtained by minimizing the objective function that was used to learn the basis via gradient descent algorithm and are given by:

$$\begin{aligned} \vec{F}(x, y) &= (F_1(x, y), \dots, F_K(x, y)) \\ &= \arg \min_{\vec{a}} \left(\left\| P(x, y) - \sum_{j=1}^K \vec{w}_j a_j \right\| + \lambda S(\vec{a}) \right) \end{aligned}$$

... (5)

2. APPLYING LOCAL MAXIMUM OPERATION

To Implement local shift invariance they divided the input image I into a set of regular, nonoverlapping regions R_i , where $i = 1, \dots, M^2$ and took the maximum and minimum of each region with respect to each coefficient image.

$$\begin{aligned} F_j^{\max}(R_i) &= \max_{x, y \in R_i} F_j(x, y). \\ F_j^{\min}(R_i) &= \min_{x, y \in R_i} F_j(x, y). \end{aligned} \quad .. (6)$$

Finally, from those two steps the authors were able to obtain the final feature vector of each input image that can be fed to any classifier. The final feature vector consists of the maximum and minimum values of all regions with respect to all coefficient images as:

$$\begin{aligned} f_I &= (F_1^{\max}(R_1), \dots, F_1^{\max}(R_{M^2}), \dots, \\ &\quad F_K^{\max}(R_1), \dots, F_K^{\max}(R_{M^2}), \\ &\quad F_1^{\min}(R_1), \dots, F_1^{\min}(R_{M^2}), \dots, \\ &\quad F_K^{\min}(R_1), \dots, F_K^{\min}(R_{M^2})). \end{aligned} \quad .. (7)$$

They implemented this method on the widely used benchmark for digit recognition MNIST dataset. The dataset consists of 60,000 training and 10,000 test images of handwritten digits of size 28 x 28 pixels. As shown in Fig. 4. They used image patches of size 13 x 13 pixels with basis functions of 169 same as the number of pixels of the patch image. Using a region R_i of size 9 x 9 pixels in the image and coefficient image of size equal to the input images they were able to obtain the final feature vector f_I with a size of $(9 \times 169 \times 2) = 3042$ from a MNIST input image of size 784.

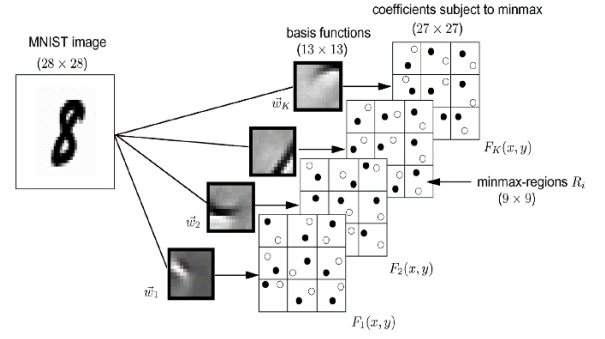


Fig. 4. Schematic view of feature extraction method proposed on the study. (adapted from [8])

The authors trained SVM on the resulting feature vectors and obtained state-of-the-art classification performance in the digit recognition. In order to compare their method with other feature extraction methods they have implemented Gabor wavelets and principal component analysis (PCA) for feature extraction (Fig. 5) on the same MNIST dataset and have fed to the same SVM classifier to compare different classification performance obtained with these methods.

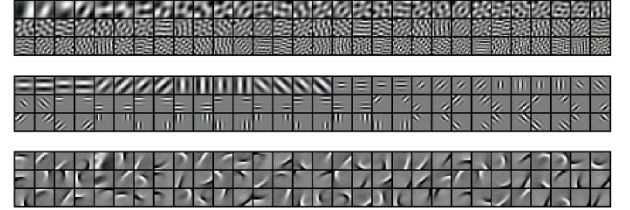


Fig. 5. Basis functions learned from top to bottom PCA, Gabor wavelets, and sparse on the MNIST dataset. (adapted from [8])

The classifier they used is a standard 2-norm soft margin SVM with Gaussian kernels. The results they obtained with their methods is shown in Table 1. here Below.

	mean validation error rate	#SVs per digit class										error rate on MNIST test set
		0	1	2	3	4	5	6	7	8	9	
raw data	2.95%(±0.17)	9785	5994	13241	13244	10976	12989	9842	10429	14015	12033	1.42%
PCA	1.29%(±0.08)	3293	2293	4580	4731	4041	4223	3676	3747	5246	4930	0.80%
Gabors	1.24%(±0.10)	4298	2970	5959	5690	4647	5371	4527	4986	6582	5630	0.71%
Sparsenet	1.00%(±0.09)	2698	1812	3667	3650	2833	3505	2770	3089	4216	3610	0.59%

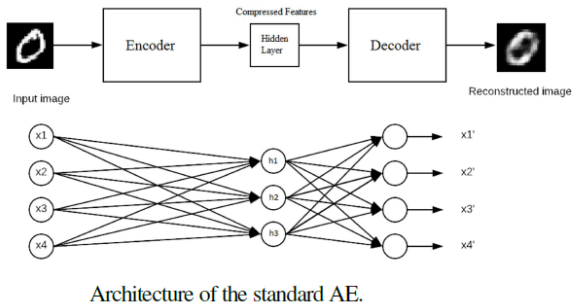
Table.1. SVM Results. (adapted from [8])

The result showed that they were able to recognize hand written digit of the MNIST dataset using the proposed method of Sparsenet algorithm with local maxima with an error rate of 0.59 %. The result obtained using their method is significantly better than the results obtained with Gabor wavelets and PCA. But as the authors indicated the proposed method requires to solve the optimization problem via gradient descent for each patch of a given input image, therefore it is not as efficient as those of like PCA and Gabor wavelet feature extraction from a computation complexity point of view.

Finally, the result shows the method they proposed is on a competitive benchmark with current state-of-the-art results. Currently The best results in this ranking are those obtained by Cireşan et al. [9], which reported an error rate of 0.23% using committees of 7 and 35 neural networks and data augmentation in all cases.

VIII.3 AUTO-ENCODER VARIANTS FOR SOLVING HANDWRITTEN DIGITS CLASSIFICATION PROBLEM

On this paper [10] the authors proposed a standard AE and, performed the comparative study of three popular AE variants: Sparse AE (SAE), denoising AE (DAE), and contractive AE (CAE) on the basis of their mathematical modeling and experiments. Then they evaluated these variants on the basis of the MNIST benchmark handwritten digit dataset for classification problem. As explained in section IV of this paper an autoencoder is a type of neural network used for unsupervised learning, particularly for dimensionality reduction and feature learning. It consists of an encoder and decoder, and the goal is to learn a compact representation (encoding) of the input data. The following Figure shows the complete structure and step by step working mechanism of the proposed model.



Algorithm 1. Auto-encoder

Parameters initialization

- No. of hidden layers: h
- Input feature set: $[x_1, x_2, x_3, \dots, x_n]$
- Encoding-activation function: EAF
- Decoding-activation function: DAF
- Inputs weights: W_i
- Biasness values: b_i

Encoding

- Compute encoded inputs $f(w)$ by multiplying x_n and W_i
- Compute biased inputs $f(b)$ by adding b_I to the encoded inputs
- Compute $f(x)$ using (a) by applying $f(w)$ and $f(b)$

Decoding

- Compute decoded outputs $f(w')$ by multiplying x_n and W_i
- Compute biased outputs $f(b')$ by adding b_I to the decoded outputs
- Compute $g(y)$ using (b) by applying $f(w')$ and $f(b')$

Optimization

- Optimize the value of (c)

$$y = f(x) = s_e(w_x + b_h), \dots (a)$$

$$r = g(y) = s_d(w'_y + b'_r), \dots (b)$$

$$J_{AE}(\theta) = \sum_{x \in D_i} R(x, r), \dots (c)$$

VIII.3.1 SPARSE AUTO-ENCODER

A sparse autoencoder is a variant of an autoencoder that incorporates sparsity constraints during training. The sparsity constraint encourages the auto encoder to learn a sparse representation of the input data, meaning that only a small number of neurons in the hidden layer should be activated for any given input. This can lead to more meaningful and selective feature extraction. The cost function of the SAE they proposed is:

$$J_{SAE}(\theta) = C(w, b) + \beta \sum_n x \in D_n L(x, r) + (\rho ||\rho),$$

..... (8)

Where n denotes the total number of neurons in the implicit layer and β the weight of the sparse penalty term. $C(w, b)$ denotes cost function, where w represents the wight matrix and b the deviation matrix.

VIII.3.2 DENOISING AUTO-ENCODER

A denoising autoencoder is designed to learn robust representations by training on corrupted versions of input data and reconstructing the clean data. The idea is to introduce noise to the input during training and teach the autoencoder to remove this noise when reconstructing the original, uncorrupted input. The proposed denoising autoencoder works by minimizing the following cost function.

$$\begin{aligned} J_{DAE}(\theta) &= E_{qo}(x, r) [||decode(encode(x)) - r||^2] \\ &= E_{qo}(x, r) [||W^T sigmoid(W_x + b) + c - r||^2]. \end{aligned} \dots (9)$$

First, they have a training input $x \in D_n$ corrupted by the additive Gaussian noise of covariance. Then they have encoded the corrupted version r into a hidden representation $x \in R^{d_h}$ via an affine mapping, followed by the nonlinearity of $h = encode(r) = sigmoid(w_r + b)$ where $x \in R^{d_h}$, $h \in (0, 1)^{d_h}$, W is a $d_h * d$ matrix with $b \in R^{d_h}$. Then the they have decoded the hidden layer representation h into reconstruction x^r via affine mapping such that $x^r = decode(h) = W_h^T + c$, where $c \in R^d$. While $\theta = w, b, c$ is optimized to minimize the squared reconstruction error $||x^r - x||$.

VIII.3.3 CONTRACTIVE AUTO-ENCODER

A contractive autoencoder is another variant of autoencoder that includes a penalty term during training to enforce a smooth and stable mapping from inputs to encoded representations. This penalty term helps make the autoencoder

Fig.6. Proposed Auto Encoder. (adapted from [10])

less sensitive to small variations in the input space. The CAE is given on their paper by the following.

$$J_{CAE}(\theta) = \sum_{x \in D_n} L(x, r) + \frac{1}{2}(\lambda \|f(x)\|_F^2), \dots (10)$$

Where $f(x)$ denotes the Jacobian matrix of encoder f at x . The penalty is the Frobenius norm of the encoder Jacobian. The penalty term encourages the Jacobian matrix to be close to the identity matrix, indicating that small changes in the input space lead to small changes in the encoded representation. This helps create a smooth and stable mapping.

On their experiment to evaluate and then compare the feature-learning abilities of the three AEs variants, they assumed three of them have the same architecture with one input layer, two hidden layers, and a Softmax regression classifier in the last layer for final classification. They have used the three types of MNIST datasets, the basic dataset of MNIST that comprises the standard MNIST image without any changes, the bg-rand variant dataset, on which a background is randomly added to the digit image, and the bg-img-rot dataset which variation factor of rotating digits and addition of random background is merged in a single dataset. See Fig.7,8. below.



Fig.7. MNIST random-noise background digits (bg-rand). (adapted from [10])



Fig.8. MNIST rotation and image background digits (bg-img-rot). (adapted from [10])

They have verified and validated the results of AE variants as in the following table.

Approach	basic			bg-rand			bg-img-rot		
	Execution time	Training error	Test error	Execution time	Training error	Test error	Execution time	Training error	Test error
SAE + Softmax	24m 25s	5.59	13.54	31m 25s	8.92	21.54	40m 25s	11.87	26.58
DAE + Softmax	31m 20s	7.73	15.78	37m 20s	10.38	21.78	45m 20s	17.73	28.72
CAE + Softmax	28m 55s	3.35	11.65	32m 55s	7.58	20.65	37m 55s	11.05	24.78

Table 2. Performance evaluation of SAE, DAE, and CAE based on MNIST benchmark datasets (adapted from [10])

From their results although SAE had faster execution time, CAE outperformed SAE and DAE with much less training and testing errors.

V. CONCLUSION

In conclusion, on this paper I have explored the pivotal role of sparse coding and autoencoders as transformative tools in the field of digital image processing. Through an exploration

of their underlying principles and applications, it becomes evident that these techniques have the potential to revolutionize how we analyze and interpret images. The demonstrated success in multimedia, remote sensing, and computer vision applications underscores the versatility and efficacy of sparse coding and autoencoders. Additionally, the focused discussion on their application in handwritten digit recognition highlights their adaptability to diverse domains, particularly in the realm of pattern recognition. The insights gained from my exploration contribute to a deeper understanding of the transformative impact of sparse coding and autoencoders on digital image processing.

REFERENCES

- [1] Ma, Y., & Wang, Y. (2018). Sparse coding and its applications in image and vision computing: A review. *Neurocomputing*, 289, 35-48.
- [2] Gonzalez R., Woods R. *Digital Image Processing* 4ed 2018
- [3] Vincent, P., Larochelle, H., Bengio, Y., & Manzagol, P. A. (2008). Extracting and composing robust features with denoising autoencoders. In *Proceedings of the 25th international conference on Machine learning* (pp. 1096-1103).
- [4] Wang, Z., & Gupta, R. (2018). Sparse coding and deep learning in multimedia processing: A review. *Multimedia Tools and Applications*, 77(1), 193-216.
- [5] Zhang, J., & Ma, Y. (2018). Sparse coding and deep learning in remote sensing image analysis: A review. *Remote Sensing*, 10(10), 1477.
- [6] Alejandro Baldominos, Yago Saez and Pedro Isasi (2019). A Survey of Handwritten Character Recognition with MNIST and EMNIST. *Appl. Sci.* 2019, 9(15), 3169; doi/10.3390/app9153169.
- [7] Rohan Sethi1, Ila Kaushik2. (2020). Hand Written Digit Recognition using Machine Learning. *IEEE 9th International Conference on Communication Systems and Network Technologies (CSNT)*. DOI: 10.1109/CSNT48778.2020.911574.
- [8] Kai Labusch, Erhardt Barth, and Thomas Martinetz. Simple Method for High-Performance Digit Recognition Based on Sparse Coding, *IEEE transactions on neural networks*, vol. 19, no. 11, november 2008.
- [9] Cireřan, D.C.; Meier, U.; Schmidhuber, J. Multi-column deep neural networks for image classification. In *Proceedings of the 2012 IEEE Conference on Computer Vision and Pattern Recognition*, Providence, RI, USA, 16–21 June 2012; pp. 3642–3649.
- [10] Muhammad Aamir, Nazri Mohd Nawi, Hairulnizam Bin Mahdin, Rashid Naseem, and Muhammad Zulqarnain. Auto-Encoder Variants for Solving Handwritten Digits Classification Problem. *International Journal of Fuzzy Logic and Intelligent Systems*, vol. 20, no. 1, March 2020. doi.org/10.5391/IJFIS.2020.20.1.8